

Lab 2 — Task 1: Containerize a Sample Application Using Docker

Akash Singh Sagar | Repository: github.com/akashsinghsagar/devops-lab2

Objective

Containerize a small Flask application with Docker: write the application and a Dockerfile, build an image, run it as a container, and verify it responds correctly and that its layers were built as designed.

Step 1 — Application source

A small Flask application (app.py) is written as the containerized workload. It exposes "/", which returns a JSON payload with the container hostname, a UTC timestamp, and a visit counter persisted to a file under DATA_DIR (defaulting to /data), and "/health", a simple liveness check.

```
app.py

import os, socket
from datetime import datetime, timezone
from flask import Flask, jsonify

app = Flask(__name__)
DATA_DIR = os.environ.get("DATA_DIR", "/data")
COUNTER_FILE = os.path.join(DATA_DIR, "counter.txt")

def read_counter():
    try:
        with open(COUNTER_FILE, "r") as f:
            return int(f.read().strip() or 0)
    except FileNotFoundError:
        return 0

def write_counter(value):
    os.makedirs(DATA_DIR, exist_ok=True)
    with open(COUNTER_FILE, "w") as f:
        f.write(str(value))

@app.route("/")
def index():
    count = read_counter() + 1
    write_counter(count)
    return jsonify({
        "message": "Hello from the Lab 2 containerized Python app!",
        "hostname": socket.gethostname(),
        "visits_recorded_in_this_container": count,
        "data_dir": DATA_DIR,
        "time_utc": datetime.now(timezone.utc).isoformat(),
    })

@app.route("/health")
def health():
    return jsonify({"status": "ok"})

if __name__ == "__main__":
```

```
app.run(host="0.0.0.0", port=5000)
```

Step 2 — Dockerfile

The Dockerfile builds on python:3.12-slim. The dependency manifest (requirements.txt) is copied and installed before the application source, so the pip-install layer stays cached across rebuilds when only app.py changes.

```
# Layer 1: base image
FROM python:3.12-slim

# Layer 2: metadata / working directory
WORKDIR /app

# Layer 3: dependency manifest copied first so pip install is cached
# independently of application code changes
COPY requirements.txt .

# Layer 4: install dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Layer 5: copy application source
COPY app.py .

# Layer 6: create data directory used for storage demo
RUN mkdir -p /data

# Metadata: document the port the app listens on
EXPOSE 5000

# Layer 7: default command to run the container
CMD ["python", "app.py"]
```

Step 3 — Build the image

The image is built and tagged lab2-python-app:1.0, then confirmed in the local image list.

```
cd lab2-docker/app
docker build -t lab2-python-app:1.0 .
docker images lab2-python-app
```

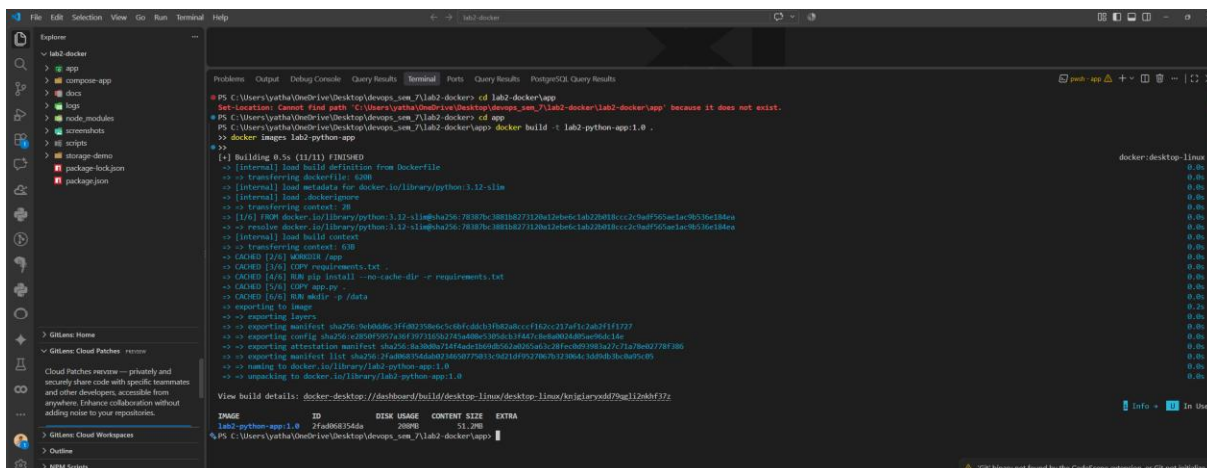


Figure 1 — Image build completes in 7 steps (BuildKit); lab2-python-app:1.0 is 208 MB with a 51.2 MB compressed layer size.

Step 4 — Run the container and verify

The image is run as a detached container publishing container port 5000 to host port 5000. Both routes are exercised with curl, and docker history / docker logs are used to inspect the resulting layer stack and runtime output.

```
docker run -d -p 5000:5000 --name lab2-app lab2-python-app:1.0
curl.exe -s http://localhost:5000/
curl.exe -s http://localhost:5000/health
docker history lab2-python-app:1.0
docker logs lab2-app
```

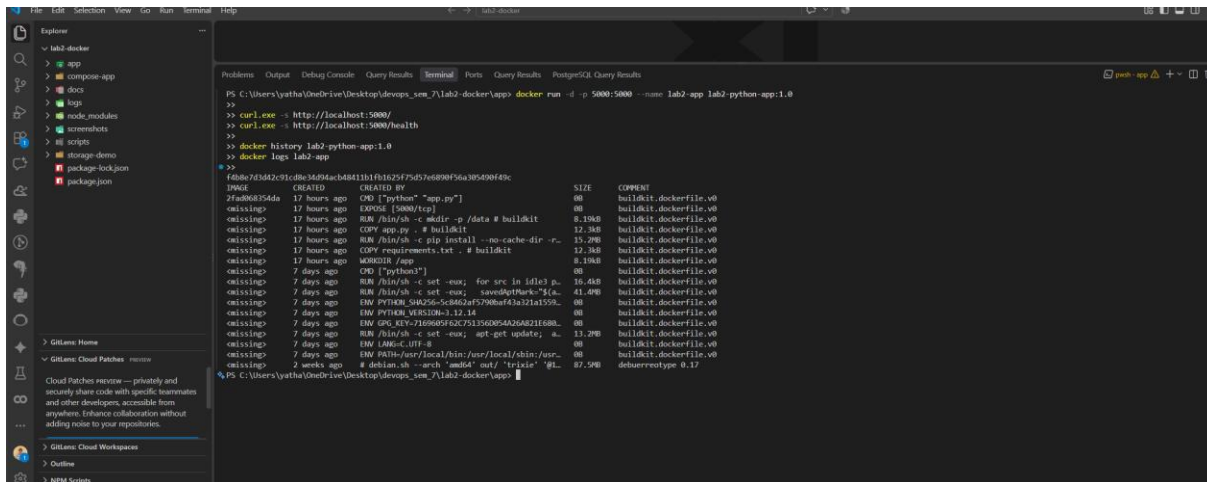


Figure 2 — docker history shows the seven authored layers (mkdir, COPY app.py, pip install, COPY requirements.txt, WORKDIR) stacked on the python:3.12-slim base layers.

Result

Image lab2-python-app:1.0 was built successfully (208 MB, 51.2 MB compressed) with dependency-caching layers in the correct order. The container served both / and /health on port 5000, and docker history confirmed each Dockerfile instruction produced its own layer as designed. Task 1 is complete.